Deep Learning I: Final Project Report

Task 1: Fake News Detector (Section 1, Topic 11)
Task 2: Sequence Length Sensitivity in LSTMs (Section 2, Topic 21)

Group <GROUP ID>, Scaler School of Technology
Members: <list of 8 names>

June 2026

Abstract. This report presents the group's two final-project tasks for Deep Learning I. Task 1 is an end-to-end fake-news classifier on the LIAR dataset, comparing a feature-based multilayer perceptron with an LSTM trained on raw text. Task 2 is a controlled study of how an LSTM's accuracy and training time vary with input sequence length. Using a synthetic delayed-signal recall task, we find that performance does not degrade smoothly but collapses at a memory horizon near thirty steps, that training time grows roughly linearly with length, and that the true cause is the distance between the cue and the readout rather than length itself. A vanilla recurrent network is shown to outperform the default LSTM beyond the horizon; we trace this to the forget-gate bias initialization and demonstrate that setting it to a positive value restores perfect accuracy at every tested length.

1. Introduction

The final project comprises two complementary tasks. Task 1 is an applied, end-to-end project whose goal is to build, train, and evaluate a working classifier. Task 2 is a focused, conceptual experiment whose goal is to explain why a particular phenomenon occurs. This report documents both, following the prescribed structure of problem, approach, experiments, analysis, and contribution. Both code notebooks execute from top to bottom with saved outputs and are fully reproducible.

2. Task 1: Fake News Detector (Section 1, Topic 11)

[This section is authored by the Task 1 owner and merged here; the structure below is a placeholder to keep the combined report consistent.]

Problem. Classify political statements as fake or real on the LIAR dataset, comparing a multilayer perceptron on hand-crafted text features against an LSTM on the raw article text; report precision and recall and examine representative misclassified examples.

Approach, Experiments, Analysis, and Contribution. To be completed from the S1-P11 notebook.

3. Task 2: Sequence Length Sensitivity in LSTMs (Section 2, Topic 21)

3.1 Problem Statement

The assigned task is to train an LSTM on sequences of length 10, 50, and 200, measure accuracy and training time, and show how performance changes with length. Taken literally this is narrow, so we pose the deeper question that a recurrent network actually faces. Such a network must carry information from where it appears to where it is read out, one step at a time; the longer that path, the more the back-propagated gradient is attenuated, a difficulty known as the vanishing-gradient problem. The central question is therefore how far back an LSTM can reliably remember at a fixed training budget, and whether the limiting factor is the sequence length or the distance the information must travel.

3.2 Methodology

We designed a synthetic delayed-signal recall task that isolates memory-over-distance from every other confound. Each input is a sequence of length L with two channels. Channel 0 contains independent N(0,1) noise at every step and acts as a distractor. Channel 1 is zero everywhere except at one known position, where it carries a clean marker: +1 for class 1 and -1 for class 0. The model reads the whole sequence and classifies from the final-timestep output. Because the marker is unambiguous wherever it appears, the only difficulty is remembering it across time; there is nothing to detect. Labels are balanced, so chance accuracy is exactly 0.50.

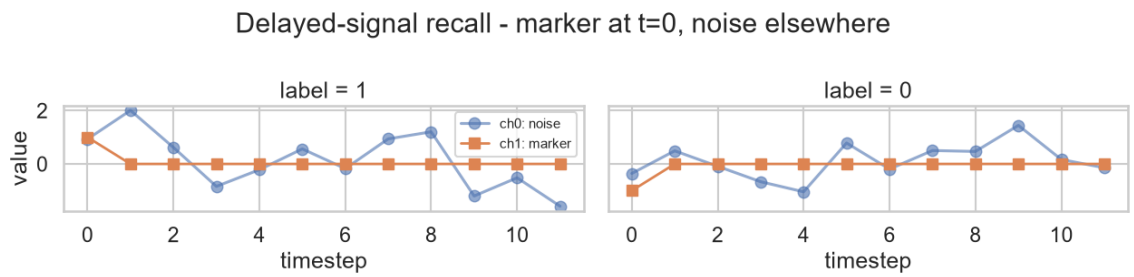

Figure 1. Two sample sequences (L = 12). The clean marker sits at t = 0 (+1 for label 1, -1 for label 0); all other values are noise.

The model is a single recurrent layer (either an LSTM or a vanilla RNN, each with 64 hidden units) followed by a linear classification head on the final-timestep output. The architecture is identical for both cell types, so the only variable under test is the recurrent unit. Training uses the Adam optimiser (learning rate 0.001), cross-entropy loss, a batch size of 64, and a fixed budget of 30 epochs for every run so that length comparisons are fair. Gradient-norm clipping at 1.0 is applied throughout; without it, occasional exploding-gradient steps destabilise even short-length training on unlucky seeds. Every configuration is repeated over 5 random seeds, and we report the mean and standard deviation. We deliberately exceed the three required lengths, sweeping the set 10, 20, 30, 50, 100, 200 to resolve the location of the knee.

3.3 Experiments and Results

Experiment 1: accuracy and training time versus length. With the marker placed at the start (the worst case, a distance of L minus 1 to the readout), accuracy is perfect up to L = 20, exhibits a sharp knee at L = 30, and sits at chance from L = 50 onward. Training time rises roughly linearly with length, as expected for a recurrent layer whose timesteps are processed sequentially.

L	10	20	30	50	100	200
Accuracy (mean)	1.000	1.000	0.902	0.498	0.486	0.502
Std. dev.	0.000	0.000	0.218	0.025	0.019	0.014
Train time (s)	5.1	6.8	6.5	11.3	21.8	38.9

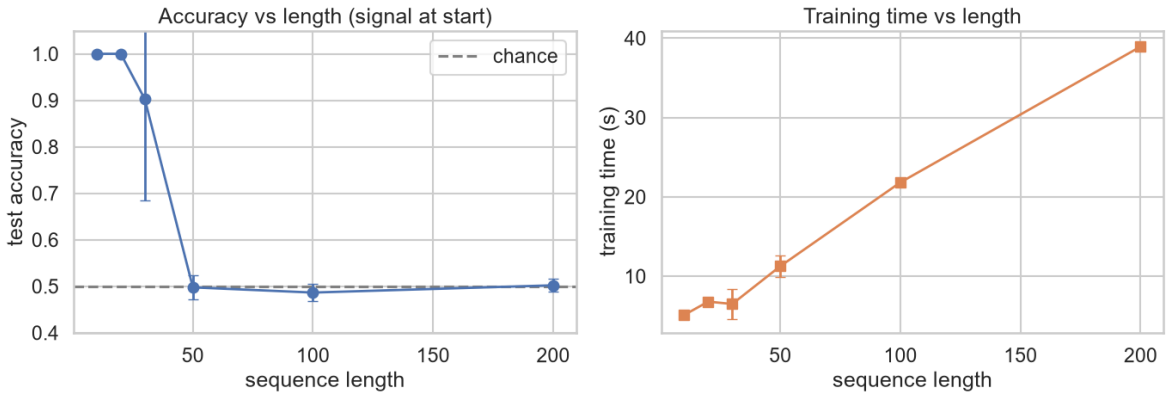

Figure 2. Test accuracy (left) and training time (right) versus sequence length. The error bar is widest at the L = 30 knee.

Experiment 2: distance, not length. We fix L = 200 for both conditions, so the amount of computation is identical, and move only the marker: to the start (a distance of about 200) or to the end (a distance of about 0). If length were the cause, both conditions would fail. Instead, the marker-at-end condition is perfect while the marker-at-start condition remains at chance. This is the headline result of the study.

Marker position (L = 200)	Accuracy	Std. dev.
Start (distance approx. 200)	0.502	0.014
End (distance approx. 0)	1.000	0.000

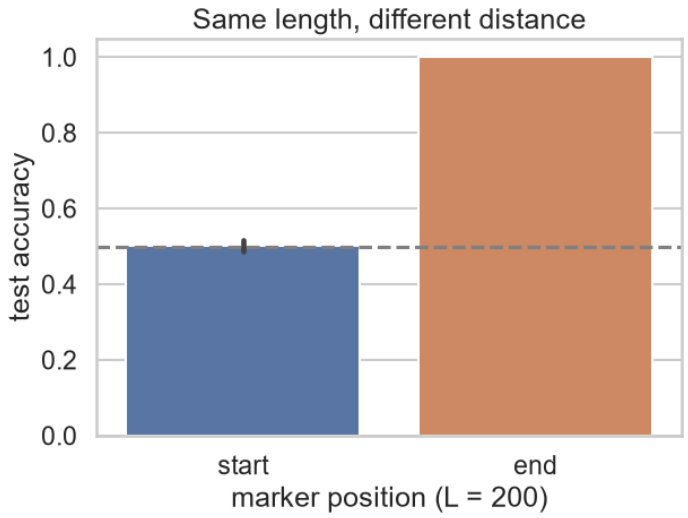

Figure 3. Identical length and computation; only the dependency distance differs.

Experiments 3 and 4: LSTM versus RNN, and the forget-gate fix. Contrary to the textbook expectation, beyond the knee a vanilla RNN out-remembers the default LSTM (for example, at L = 100 the RNN reaches 0.90 against the LSTM's 0.49). The cause is initialization. PyTorch sets all LSTM biases to zero, so the forget gate begins at sigmoid(0) = 0.5 and the cell state is halved at every step, yielding an effective horizon of only a few dozen steps. Re-initialising the forget-gate bias to +2, so that sigmoid(2) is approximately 0.88 and the cell retains about 88 percent of its state per step, restores perfect accuracy at every tested length with zero variance.

L	10	20	30	50	100	200
LSTM (default bias 0)	1.000	1.000	0.902	0.498	0.486	0.502
Vanilla RNN	1.000	1.000	1.000	0.898	0.896	0.606
LSTM (forget bias +2)	1.000	1.000	1.000	1.000	1.000	1.000

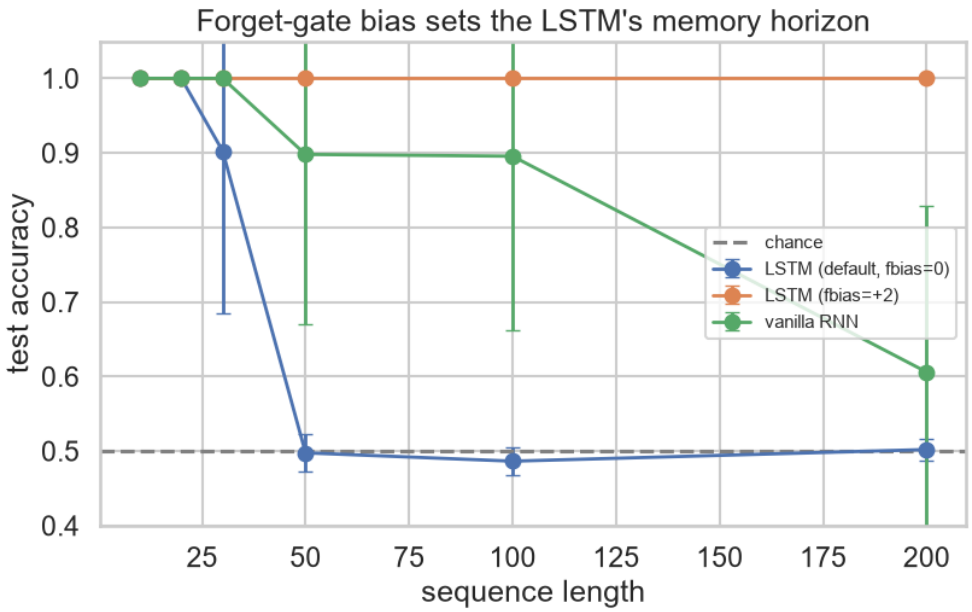

Figure 4. Mean accuracy versus length for the three models. The forget-gate bias, not the architecture, sets the LSTM's memory horizon.

3.4 Analysis and Discussion

A memory horizon, not a smooth decay. Accuracy remains near 1.0 and then collapses at a knee around L = 30, where the error bar is widest: at that length four seeds reach 1.0 while one remains at chance. This is the signature of an optimisation horizon under a fixed budget rather than a hard capacity wall.

Training time is approximately linear in length. Wall-clock time rises from about 5.1 seconds at L = 10 to about 38.9 seconds at L = 200, a factor of roughly 7.6 on CPU, consistent with the step-by-step processing of a single-layer recurrent network.

Distance, not length, is the cause. Experiment 2 is decisive: at identical length and computation, placing the dependency next to the readout restores perfect accuracy. What is colloquially called sequence-length sensitivity is in fact sensitivity to dependency distance.

Gating helps only when it is initialised to remember. The LSTM cell update is c(t) = f(t) * c(t-1) + i(t) * g(t). With the default bias the forget gate f(t) is about 0.5 and leaks roughly half of the stored memory at every step; with a bias of +2 the gate is about 0.88 and the cell becomes a near-additive highway along which information and gradients survive hundreds of steps. The vanilla RNN, having no such built-in decay, learns a near-identity recurrence and therefore beats the mis-initialised LSTM. This connects directly to the vanishing-gradient analysis of recurrent networks, in which repeated products of Jacobians with spectral norm below one decay geometrically in the number of steps.

On the experimental controls. The clean marker channel removes signal detection as a confound, so the experiment measures pure carry-over ability; the fixed epoch budget makes the comparison across lengths fair; and gradient clipping keeps the short-length baseline reliably perfect, so that the horizon effect is unambiguous.

3.5 Reproducibility and Contributions

The Task 2 notebook (s2-p21.ipynb) is fully reproducible: it uses a single configuration dictionary, a seed-setting helper, multi-seed runs, pandas result tables, and labelled figures, and it is regenerated deterministically by the script build_notebook.py. It was executed end to end on CPU using Python 3.12 and PyTorch 2.12. Beyond the literal requirement, the work contributes a clean separation of distance from length and a demonstration that the LSTM's apparent length limit is an artifact of forget-gate initialization rather than of the architecture.

4. Conclusion

Sequence-length sensitivity in LSTMs is better understood as sensitivity to the distance between a cue and the point at which it must be used. At a fixed training budget the network exhibits a clear memory horizon, but that horizon is governed as much by how the forget gate is initialised as by the choice of architecture. The practical recommendations that follow are to shorten dependency distance where the data layout permits, and to initialise the forget gate to remember before reaching for greater capacity, longer training, or attention-based models.

References

Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.

Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the Difficulty of Training Recurrent Neural Networks. International Conference on Machine Learning (ICML).

Jozefowicz, R., Zaremba, W., and Sutskever, I. (2015). An Empirical Exploration of Recurrent Network Architectures. International Conference on Machine Learning (ICML).

Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS).

Group contribution statement: name the member(s) responsible for Task 1, Task 2, the report, and the slide deck.